

안녕하세요, 지민수입니다.

1998.08.28

entersuk1998@naver.com / 01025955586

서비스 운영 관점에서 문제를 봅니다.

저는 보이는 기능보다 서비스가 실제로 운영될 때 어떤 문제가 생기는지를 먼저 봅니다. 업로드 상태, 조회 구조, 운영 지표처럼 사용자는 잘 느끼지 못하지만 서비스 품질을 좌우하는 기준은 결국 백엔드 구조에서 나온다고 생각합니다. 그래서 프로젝트에서도 기능 구현에 그치지 않고, 데이터 흐름과 상태 관리, 운영 기준까지 함께 정리하는 방식으로 문제를 풀어 왔습니다.

왜 막히는지 끝까지 확인하고 다시 고칩니다.

한 번에 잘 풀어내기보다, 원인을 먼저 확인하고 설명 가능한 방식으로 다시 정리하려고 합니다. 파워리프팅 3대 700과 국토종주 2회 등을 준비하며 익힌 반복과 복기의 습관도 개발에 그대로 가져오고 있으며, 이 태도가 결국 오래 버티는 구조를 만드는 데 가장 중요하다고 생각합니다.

협업과 도구 활용은 기준으로 통제합니다.

회의는 아젠다 기반으로 운영하고, ADR과 PR 규칙을 통해 작업 기준을 맞췄습니다. 프론트엔드와는 API 문서를 중심으로 빠르게 소통했고, AI도 단순 코드 생성 도구로 쓰기보다 문서와 규칙 안에서 통제 가능한 방식으로 활용했습니다.

희망 직무	백엔드 엔지니어
주요 기술	Java / Spring Boot / JPA Redis / MySQL / AWS S3 / RDS Docker Compose

경력

원티드랩 부트캠프 2025.09 - 2026. 03 백엔드 협업 과정 수료	팀 프로젝트 Shortudy 백엔드 개발 / 팀장
---	-----------------------------

Link	https://minji0828.github.io/minji-pop/#journey
------	---

숏터디는 짧은 학습 영상을 업로드하고 소비하는 숏폼 학습 플랫폼입니다. 학습 콘텐츠 품질 보장을 위해 AI 검수 흐름을 함께 두었고, 저는 그 정책이 반영되는 상태 관리와 조회 구조, 운영 지표 등 백엔드 영역을 중심으로 구현했습니다.

특히 업로드, 조회, 운영 지표처럼 사용자는 잘 느끼지 못하지만 서비스 품질을 좌우하는 구조를 정리하는 데 집중했습니다. 팀장으로서 협업 기준과 작업 방식을 함께 정리하며 프로젝트 전반의 구조와 흐름을 함께 다뤘습니다.

Shortudy

“숏터디는 짧지만 밀도 높은 학습 경험을 제공하는 플랫폼입니다.”

직무

백엔드 개발

주요 기술

Java / Spring Boot / JPA
Redis / MySQL / AWS S3 / RDS
Docker Compose

경험

S3 Direct Upload 기반 업로드 구조 설계
업로드 상태와 공개 상태 분리
Redis write-back 기반 조회수 집계 구조 적용
Projection 기반 조회 API 및 운영 지표 API 구현
단일 EC2 구조 한계 확인 후 RDS 분리 방향으로 배포 구조 조정

숏폼 학습 피드와 업로드 흐름을 포함한 사용자 서비스 메인 화면

인기 숏폼 🔥



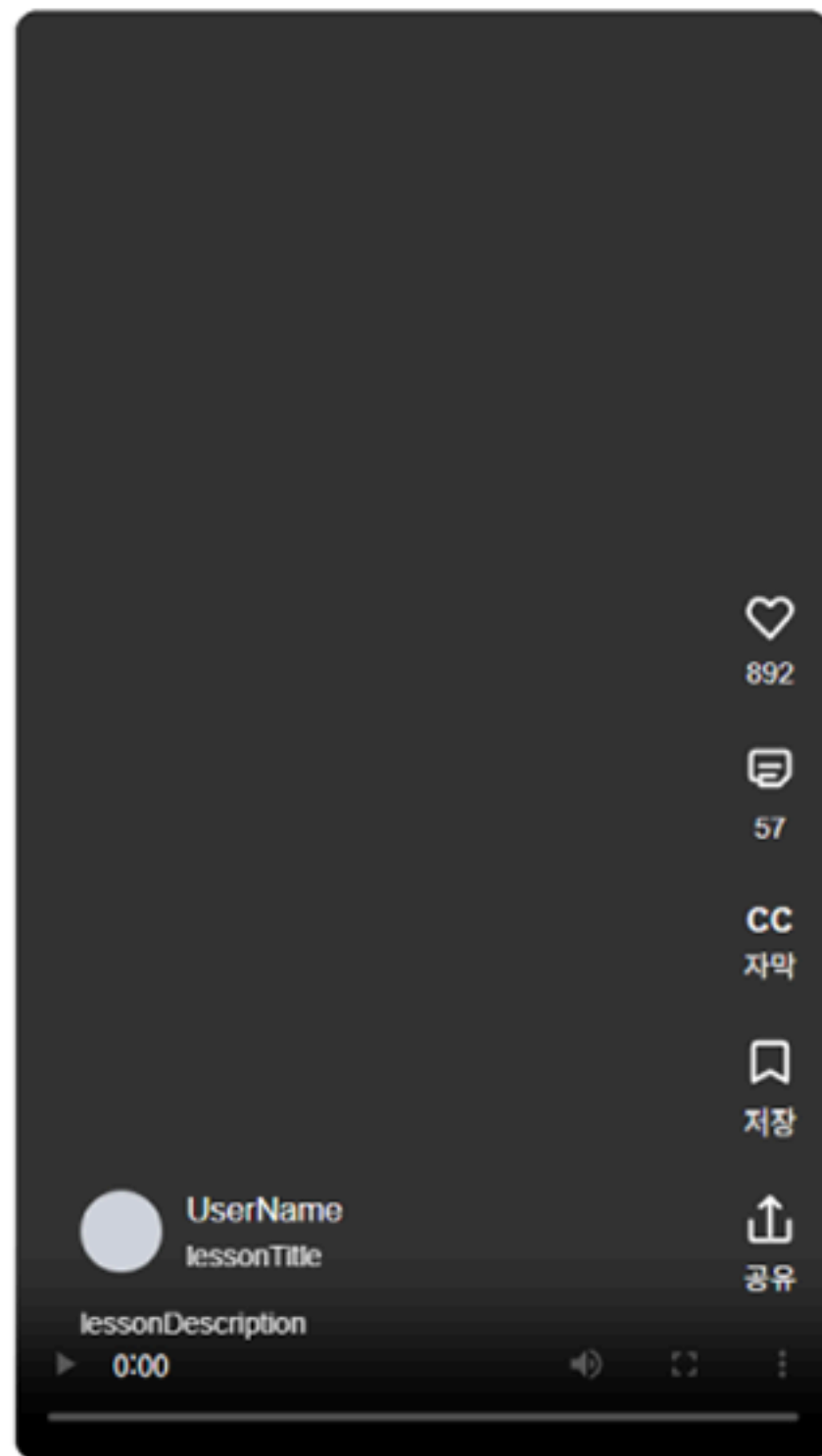
숏폼 플레이리스트



카테고리별 강의

전체보기

전체 개발 디자인 비즈니스



서버 부하를 줄이면서도 업로드 중단, 재시도, 만료 상태를 관리할 수 있는 구조를 설계했습니다.

숏 제목 • 0/40

제목을 입력하세요.

숏 설명 • 0/500

내용을 입력하세요.

카테고리 •

카테고리를 선택하세요.

키워드 (0/5) •

키워드를 영문으로 입력하세요. 예) data

동영상

썸네일

+

동영상 파일을 드래그하거나
클릭하여 업로드 하세요.
(MP4 권장)

파일 선택

등록하기

취소

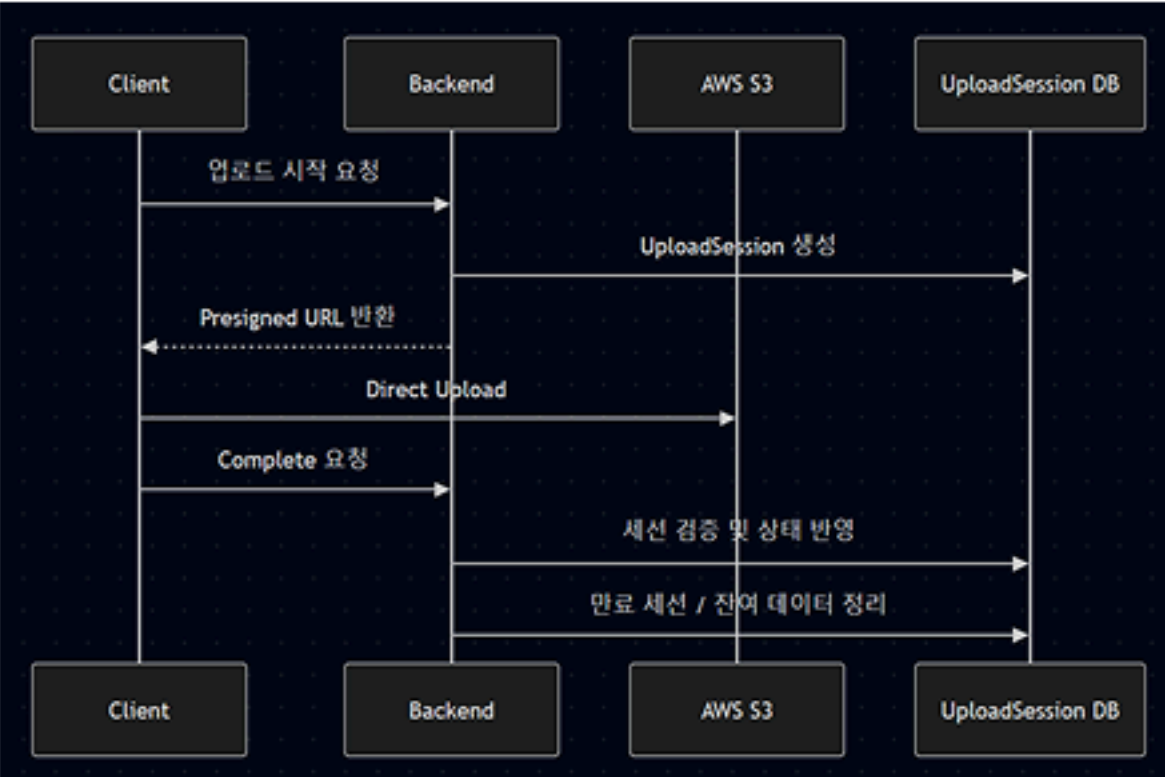
```
{
  "AllowedHeaders": [
    "*"
  ],
  "AllowedMethods": [
    "PUT",
    "GET",
    "HEAD"
  ],
  "AllowedOrigins": [
    "https://shortudy.vercel.app"
  ],
  "ExposeHeaders": [
    "ETag"
  ],
  "MaxAgeSeconds": 3000
}
```

작업 내용

- Presigned URL 발급
- 업로드 세션 분리
- 재시도 시 이전 세션 정리
- 만료/고아 데이터 정리 스케줄러

구현 내용

- Multipart Upload보다 구현 복잡도가 낮고, 서버가 파일 본문을 직접 처리하지 않아 Presigned URL 기반 Direct Upload를 선택했습니다.
- 업로드는 전송보다 중단·재시도·만료 같은 상태 관리가 더 중요하다고 판단했습니다.
- 실제 게시 엔티티인 Shorts와 업로드 수명주기 관리용 UploadSession을 분리해 구조를 설계했습니다.



검토한 대안

Multipart Upload / STS / Presigned POST

→ 당시 프로젝트 규모와 구현 복잡도를 고려해 Presigned URL 선택

어려웠던 점

- 당시에는 업로드가 실패할 수 있다는 상황, 즉 클라이언트가 완료 요청을 보내더라도 실제 S3 객체가 존재하지 않을 수 있다는 가능성까지 충분히 가정하지 못했습니다. 업로드는 정상적으로 끝난다는 전제 아래 흐름을 설계했기 때문에, complete 단계에서 실제 객체 존재 여부를 검증하는 로직은 구현하지 못했습니다. 또한 비동기 처리에 대한 이해가 부족해, 업로드 완료 이벤트를 서버가 어떤 방식으로 받아 최종 상태를 판단해야 하는지도 당시에는 설계하지 못했습니다. 이후 복기하면서 Direct Upload에서는 성공 요청 자체보다 실제 업로드 완료를 어떻게 검증할 것인가가 더 중요하다는 점을 알게 되었고, S3 이벤트 기반 후처리나 비동기 검증 구조가 필요하다고 판단했습니다.

Lessons

- 업로드 기능은 파일을 전송하는 것보다 상태를 관리하는 일이 더 중요하다는 점을 배웠습니다. 기능을 먼저 만들기보다, 중간 실패와 재시도까지 포함한 수명주기를 기준으로 구조를 나누는 것이 서비스 운영에 더 적합하다는 점을 확인했습니다.

조회수 데이터를 Redis write-back 구조로 재설계

조회수에는 강한 즉시 정합성보다 쓰기 부하 분산이 중요하다고 보고, 집계 구조를 다시 설계했습니다.

```

}

// Redis 조회수를 DB에 주기적으로 반영 (기본 1분)
@Scheduled(fixedDelayString = "${shorts.view.flush-interval-ms:60000}")
public void flushViewCounts() {
    // 식별 기준
    shortsViewCountService.flushViewCounts();
    // 로그인: 사용자 ID
    // 비로그인: IP + User-Agent 해시
}

```

```

// 중복 조회 방지를 위한 방문자 키 등록
public boolean markUniqueView(Long shortId, String visitorId, Duration ttl) {
    String key = UNIQUE_KEY_PREFIX + shortId + ":" + visitorId;
    Boolean created = valueOperations.setIfAbsent(key, "1", ttl);
    return Boolean.TRUE.equals(created);
}

```

```

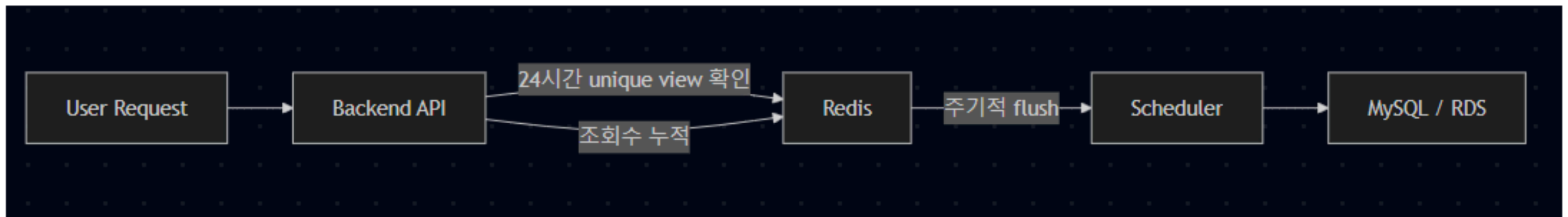
// 조회수 증가 처리
@Transactional
public void increaseViewCount(Long shortId, String visitorId) {
    if (shortId == null || shortId <= 0) {
        throw new BaseException(ErrorCode.INVALID_INPUT, "shortId: 값이 올바르지 않음");
    }
    if (visitorId == null || visitorId.isBlank()) {
        throw new BaseException(ErrorCode.INVALID_INPUT, "visitorId: 값이 올바르지 않음");
    }

    boolean isNewView = viewCountRepository.markUniqueView(shortId, visitorId);
    if (isNewView) {
        viewCountRepository.increaseViewCount(shortId);
    }
}

// Redis 누적 조회수 DB 반영
@Transactional
public void flushViewCounts() {
    Map<Long, Long> counts = viewCountRepository.findPendingViewCounts();
    if (counts.isEmpty()) {
        return;
    }

    for (Map.Entry<Long, Long> entry : counts.entrySet()) {
        shortsRepository.updateViewCount(entry.getKey(), entry.getValue());
    }
    viewCountRepository.clearPending(counts.keySet());
}

```



작업 내용

- Redis INCR 기반 조회수 누적
- 주기적 MySQL 반영 구조 적용
- 24시간 unique view 정책 구현
- 로그인/비로그인 사용자 구분 집계

구현 내용

- 기존에는 조회가 발생할 때마다 MySQL row를 직접 갱신하는 구조였고, 같은 row에 반복 update가 몰리면 병목이 될 수 있다고 판단했습니다. 반면 조회수는 결제 데이터처럼 강한 즉시 정합성이 절대적인 값은 아니라고 봤습니다. 그래서 Redis에 먼저 누적하고 일정 주기로 MySQL에 반영하는 write-back 구조를 적용해 쓰기 부담을 분산하도록 설계했습니다.

어려웠던 점

가장 어려웠던 점은 조회수의 집계 기준과 보존 기준을 함께 정하는 일이었습니다. 중복 집계를 줄이기 위해 24시간 기준 unique view 정책을 적용했고, 로그인 사용자는 사용자 ID, 비로그인 사용자는 IP와 User-Agent 조합을 해시한 fingerprint로 식별했습니다. 다만 이후 복기하면서 IP 기반 식별은 정책적으로 더 신중해야 하며, 쿠키 기반 익명 방문자 ID가 더 적절하다고 판단했습니다.

또한 Redis에 먼저 누적하는 구조에서는 Redis 장애나 캐시 유실 시 아직 DB에 반영되지 않은 pending 조회수가 사라질 수 있습니다. 당시에는 조회수를 반드시 보존해야 하는 값으로 보지 않아 AOF 같은 영속화 전략까지는 적용하지 않았고, 어느 정도 손실을 허용할지에 대한 기준도 충분히 정리하지 못했습니다. 이 경험을 통해 write-back 구조에서는 성능뿐 아니라 데이터 중요도와 손실 허용 범위까지 함께 설계해야 한다는 점을 배웠습니다.

Lessons

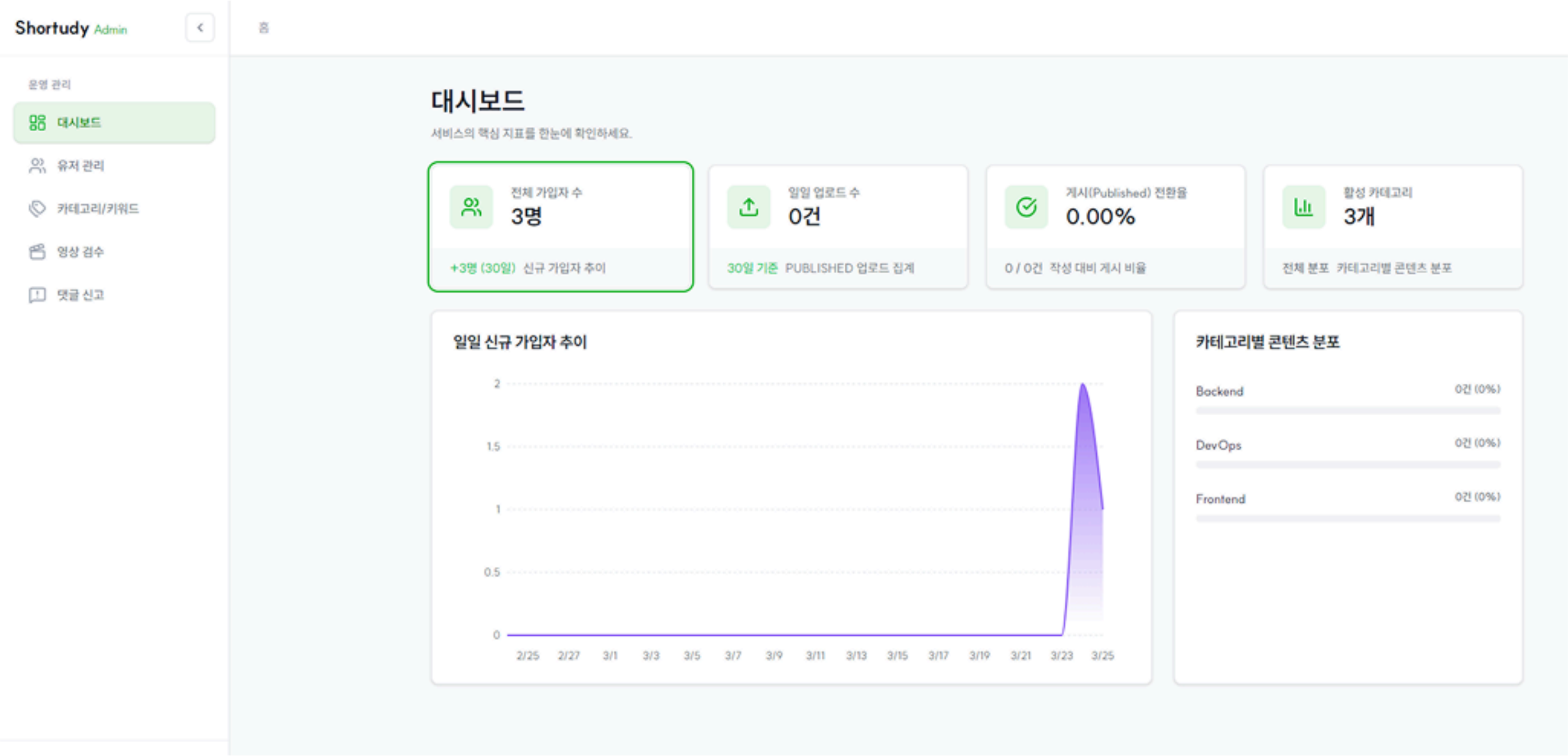
- 모든 데이터에 같은 정합성 기준을 적용하면 안 된다는 점을 배웠습니다.
- 조회수처럼 부하가 집중되는 데이터는 즉시 반영보다 데이터 성격에 맞는 저장 전략을 먼저 정하는 것이 더 중요하다는 점을 확인했습니다.
- 당시에는 조회수를 반드시 보존해야 하는 값으로 보지 않아 AOF 같은 영속화 전략까지는 적용하지 않았고, 손실 허용 범위를 더 먼저 정의했어야 한다는 점을 복기했습니다.

핵심 판단

조회수에는 결제 데이터처럼 즉시 정합성이 절대적인 값이 아니라고 보았습니다.

→ Redis write-back 구조 선택

백오피스를 별도 운영 도구로 보고, 현재 수집 가능한 데이터부터 기준을 정해 단계적으로 지표를 구성했습니다.



Phase 1

현재 DB / 기존 로직으로 바로 수집 가능한 정보

작업 내용

- BackOffice 운영 지표 API 구현
- 게시 추세 및 전환율 계산 로직 정리
- 수집 가능 데이터 기준으로 지표 설계
- 클라이언트와 분리된 백오피스 모듈 구성

Phase 2

백엔드 로직을 추가하면 수집 가능한 정보

Phase 3

프론트엔드에서 별도 수집 후 전달이 필요한 정보

구현 내용

- 프론트엔드와 일정과 범위를 조율하면서, 백오피스는 사용자 화면과 다른 기준으로 우선순위를 잡아야 한다는 점을 알게 되었습니다. 그래서 지표를 현재 바로 수집 가능한 정보, 백엔드 로직 추가 시 수집 가능한 정보, 프론트엔드에서 별도로 수집해야 하는 정보의 세 단계로 나누고, 우선 현재 구조만으로 구현 가능한 지표부터 개발했습니다.
- 또한 운영 기능이 클라이언트 서비스에 직접 영향을 주지 않도록 DB는 공유하되 모듈은 분리했습니다. 프론트의 작업 시간이 부족하여
- 백오피스 프론트는 바이브코딩 직접 구현했으며, SSR보다 빠른 구현과 정보 전달을 우선해 최소한의 디자인 일관성을 유지하는 수준에서 구성했습니다.

어려웠던 점

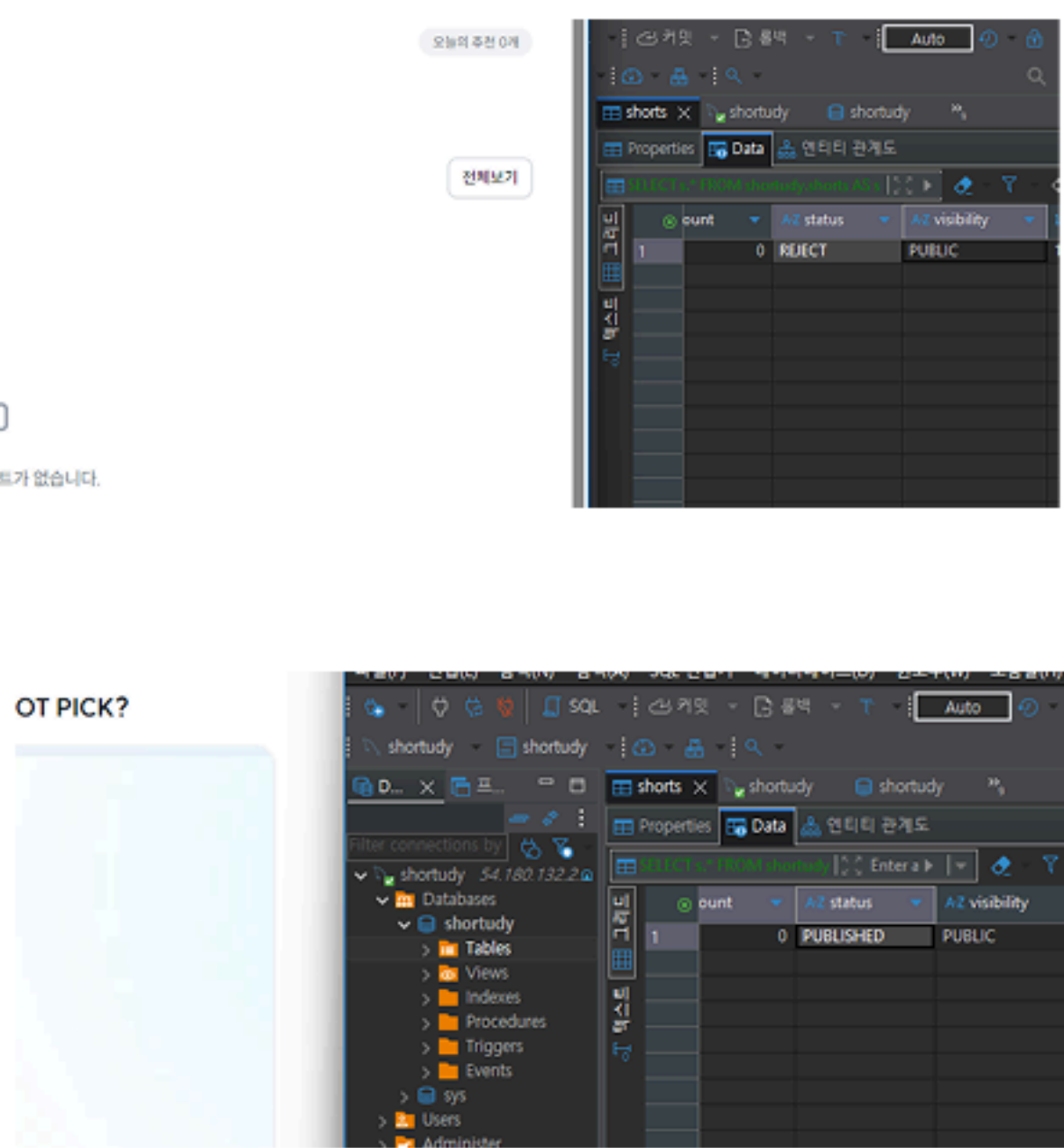
- 가장 어려웠던 점은 현재 수집 가능한 정보를 어떤 기준으로 가공해야 운영에 실제로 도움이 되는 지표가 되는지 정하는 일이었습니다. 단순히 많이 보여주는 것보다, 운영하는 입장에서 지금 꼭 필요한 정보가 무엇인지 먼저 판단해야 했습니다. 당시에는 백오피스에 어떤 정보가 진짜 필요한지에 대한 감각도 충분하지 않았기 때문에, 구현 가능한 것과 운영에 의미 있는 것을 계속 구분해 가며 범위를 조정해야 했습니다. 이 과정에서 백오피스는 화면을 하나 더 만드는 일이 아니라, 현재 서비스가 어떤 상태인지 같은 기준으로 해석할 수 있게 만드는 운영 도구라는 점을 더 분명히 이해하게 되었습니다.

Lessons

- 백오피스는 사용자 화면의 연장이 아니라 운영을 위한 별도 도구로 봐야 한다는 점을 배웠습니다. 중요한 것은 많은 지표를 한 번에 만드는 것이 아니라, 현재 수집 가능한 데이터 안에서 운영에 꼭 필요한 기준부터 정하고 단계적으로 확장하는 것이라는 점을 확인했습니다.

업로드 상태와 공개 상태 분리

업로드 완료가 곧바로 공개를 의미하지 않도록, 검수와 운영 정책이 반영되는 상태 구조를 따로 설계했습니다.



```
LEFT JOIN ShortyLike 1 ON 1.Shorty =
"WHERE s.status = 'PUBLISHED' " +
"AND s.visibility = 'PUBLIC' " +
"AND s.created_at <= '2023-07-31 23:59:59' "
```

기본적으로 PUBLISHED 와 PUBLIC 두 상태가 다 존재해야 외부 노출

영상 검수

자동 AI 검수 결과를 확인하고 게시/반려를 결정합니다.

검수 대기
1건

AI 점검
0건

게시
0건

반려
0건

필터

전체 상태

검수 요청 목록

쇼트ID	제목	영상	작성자	상태	상태 설명	AI 결과	검수시작	등록 시각	액션
14	테스트	영상 보기	지민수	대기	설명 보기	검수 대기	AI 검수 완료 후 확인 가능	3. 25. 오전 07:38	게시 반려

총 1건

이전 1 / 1 다음

작업 내용

- 업로드 상태와 게시 상태 분리
- 공개 설정과 게시 가능 조건 분리
- 조회 시 게시 가능 상태 + 공개 여부 동시 체크
- 검수 및 운영 정책이 반영되는 상태 흐름 정리

정책 요약

- 업로드 완료 != 공개 가능
- 공개 설정과 게시 가능 상태 분리
- 조회 시 두 조건을 함께 체크

구현 내용

- 프로젝트에는 학습 콘텐츠 품질을 일정 수준 이상 유지하기 위한 AI 검수 흐름이 있었기 때문에, 업로드 완료가 곧바로 공개를 의미하면 안 됐습니다. 그래서 업로드 성공과 실제 노출 가능 상태를 같은 값으로 두지 않고, 업로드 상태와 게시 상태를 분리해 관리했습니다. 조회 시에도 단순히 공개 여부만 보지 않고, 게시 가능 상태와 공개 설정을 함께 만족하는 경우에만 노출되도록 조건을 명시해 정책이 코드에 일관되게 반영되도록 정리했습니다.

어려웠던 점

- 가장 어려웠던 점은 사용자의 공개 의사와 서비스의 검수 정책을 어디서 분리해 관리할 것인지 정하는 일이었습니다. 처음에는 업로드가 끝나면 바로 공개로 이어지는 흐름으로 단순하게 볼 수도 있었지만, 프로젝트에는 학습 콘텐츠 품질을 위한 AI 검수 흐름이 있었기 때문에 업로드 완료와 실제 게시 가능 상태를 같은 의미로 다루면 안 됐습니다. 이 경험을 통해 정책이 바뀌더라도 수정 지점이 흔들리지 않게 하려면, 상태를 먼저 분리해서 설계해야 한다는 점을 배웠습니다.

Lessons

- 업로드 성공과 실제 공개 가능 상태는 다른 문제라는 점을 배웠습니다. 서비스 정책이 개입되는 기능일수록 하나의 상태값에 의미를 몰아넣기보다, 역할이 다른 상태를 분리해 두는 것이 유지보수와 운영에 더 적합하다는 점을 확인했습니다.

키워드와 실시간 조회수를 함께 조합해야 하는 읽기 경로를 단순하고 명확한 구조로 정리했습니다.

처음에는 Facade 패턴으로 조회 책임을 감싸려 했지만, 피드백 이후 쿼리 단계에서 필요한 데이터를 한 번에 가져오는 방향으로 읽기 경로를 다시 정리했습니다.

```
@Transactional(readOnly = true)
public ShortsResponse getShortsDetails(Long shortsId, Long userId) {
    // 1. 핵심 데이터 조회
    Shorts shorts = shortsService.findShortsWithDetails(shortsId);

    // 2. 부가 데이터 병렬/개별 조회 (댓글, 좋아요 등)
    long commentCount = commentCountProvider.commentCount(shortsId);
    boolean isLiked = (userId != null) &&
        shortsLikeRepository.existsByUserIdAndShortsId(userId, shortsId);

    // 3. Redis 실시간 조회수 보정 (Write-Back 반영)
    long realTimeViewCount = calculateRealTimeViewCount(shorts, shortsId);

    // 4. 단일 응답 객체 조립
    return ShortsResponse.of(shorts, commentCount, realTimeViewCount, isLiked);
}
```



```
[전체 목록 조회 쿼리]
JQuery<ShortsResponse> findResponsesByStatus(@Param("status") ShortsStatus status, @Param("userId") Long us
    "SELECT new com.example.shortudy.domain.shortcuts.dto.ShortsResponse(" +
    "s.id, s.title, s.description, s.videoUrl, s.thumbnailUrl, s.durationSec, s.status, s.visibility, " +
    "u.id, u.nickname, u.profileUrl, " +
    "c.id, c.name, " +
    "null, " +
    "s.viewCount, s.likeCount, " +
    "(SELECT count(cm) FROM Comment cm WHERE cm.shortcuts = s), " +
    "s.createdAt, s.updatedAt, " +
    "(SELECT count(1) > 0 FROM ShortsLike l WHERE l.shortcuts = s AND l.user.id = :userId)) " +
    "FROM Shorts s " +
    "JOIN s.user u " +
    "JOIN s.category c " +
    "WHERE s.status = :status AND s.visibility = 'PUBLIC'"
    )
```

초기 접근

Facade 중심으로 조회 책임을 감싸는 방향 검토

최종 접근

Projection 조회 + 배치 조회 + Redis 병합으로 읽기 경로 단순화

작업 내용

- Projection 기반 조회 적용
- 배치 조회로 키워드 로딩
- Redis pending 값 병합
- Query Service로 읽기 책임 분리

구현 내용

- 처음에는 여러 조회 책임을 Facade 패턴으로 감싸는 방향을 고려했습니다. 하지만 피드백을 통해 이 문제의 핵심은 패턴 적용 자체보다, 필요한 데이터를 쿼리 단계에서 한 번에 가져오고 N+1을 줄이며 응답 조립을 단순하게 만드는 데 있다는 점을 다시 보게 되었습니다. 그래서 기본 응답은 projection으로 직접 조회하고, 키워드는 배치 조회로 채우며, 실시간 조회수는 Redis 값을 병합하는 방식으로 읽기 경로를 정리했습니다.

어려웠던 점

가장 어려웠던 점은 실제 운영 환경에서 어떤 조회 구조가 더 비용이 적고 효율적인지 정량적으로 판단할 근거가 부족했다는 점이었습니다. 당시에는 운영 트래픽이나 응답 시간, 쿼리 비용 같은 지표가 충분히 쌓여 있지 않아 어떤 방식이 가장 낫다고 확신하기 어려웠습니다. 처음에는 Facade 패턴으로 조회 책임을 감싸는 방향도 고려했지만, 피드백을 통해 이 문제의 핵심은 패턴 자체보다 필요한 데이터를 쿼리 단계에서 한 번에 가져오고 응답 조립을 단순하게 만드는 데 있다는 점을 다시 보게 되었습니다. 그래서 당시 확인 가능한 범위 안에서 projection 조회, 배치 조회, Redis 값 병합처럼 구조를 더 단순하게 가져가는 방향으로 정리했습니다.

Lessons

- 패턴은 구조를 설명하는 도구일 뿐, 문제를 대신 해결해 주지는 않는다는 점을 배웠습니다. 조회 성능과 유지보수성이 중요한 영역에서는 패턴의 형태보다 필요한 데이터를 단순하고 명확하게 가져오는 구조를 먼저 만드는 것이 더 중요하다는 점을 확인했습니다.